LexAI Complete Technical Master Guide

Project: LexAI – AI-Powered Legal Assistant & Contract Intelligence Platform
Note: This guide is generated based on a direct inspection of the actual LexAI source code. All routes, database schemas, and AI pipelines reflect the real implementation.

MODULE 1: DATABASE AND MYSQL MASTERY

A. MySQL Fundamentals

- **Database:** An organized collection of data. In LexAI, we use a relational database where data is stored in tables (like Excel spreadsheets) connected by relationships.
- **Primary Key (PK):** A unique ID for every record (e.g., `user.id = 1`).
- **Foreign Key (FK):** A column that links to another table's Primary Key. For example, `document.user_id` links to `user.id`, proving who owns the document.

B. Actual Database Schema (ER Diagram Logic)

LexAI uses 6 core tables defined in `models/database_models.py`: 1. **Users:** `id` (PK), `name`, `email` (Unique), `password_hash`, `role`. 2. **Documents:** `id` (PK), `user_id` (FK -> Users), `filename`, `file_type`, `status`, `extracted_text`. (One-to-Many: 1 User has Many Documents). 3. **DocumentAnalysis:** `id` (PK), `document_id` (FK -> Documents), `summary` (JSON), `risks` (JSON). (One-to-One: 1 Document has 1 Analysis). 4. **Conversations:** `id` (PK), `user_id` (FK), `document_id` (FK). Tracks chat sessions. 5. **Messages:** `id` (PK), `conversation_id` (FK -> Conversations), `sender` (user/ai), `message`. 6. **KnowledgeBase:** `id` (PK), `title`, `content`, `category`. (Curated by Admins).

C. SQLAlchemy Explained

LexAI uses **Flask-SQLAlchemy** (an ORM - Object Relational Mapper). Instead of writing raw SQL strings, you write Python classes. * **Code Excerpt:** `new_doc = Document(user_id=user_id, filename=safe_filename)` * **Saving to DB:** `db.session.add(new_doc)` followed by `db.session.commit()`. * **Analogy:** The ORM

is an automatic translator. You speak Python, and it translates your commands into SQL for MySQL to understand.

D. SQL Practical Lab (Safe Queries)

To manually inspect your MySQL database via command line: * `SHOW TABLES;` -> Lists all tables. * `DESCRIBE users;` -> Shows the columns in the Users table. * `SELECT email, role FROM users;` -> Fetches all user emails. * *(Hypothetical Update)*: `UPDATE users SET role='admin' WHERE id=1;`

MODULE 2: AI, GEMINI API, CHROMADB AND RAG

A. Gemini API Integration

LexAI uses the official `google-genai` SDK. * **Initialization:** `client = genai.Client(api_key=api_key)` is called in `rag_service.py`. * **Model:** Uses `gemini-3.6-flash` (or the model specified in `AI_MODEL` config). * **Handling Errors:** Wrapped in a `try/except` block. If Gemini throws a `429 Quota Exceeded` error, LexAI triggers `construct_fallback_response()` to serve raw Knowledge Base data without crashing.

B. Document Processing & Chunking

Found in `document_service.py`: 1. **Extraction:** PyMuPDF is used for PDFs; `python-docx` is used for Word files. 2. **Chunking (`embedding_service.py`):** The extracted text is split into chunks of **1000 words** with an overlap of **150 words** to ensure sentences aren't cut in half.

C. Embeddings

Found in `embedding_service.py`: * **Model:** LexAI uses the local `SentenceTransformer('all-MiniLM-L6-v2')`. * **Concept:** It converts text into an array of 384 numbers. Similar texts end up mathematically close to each other in vector space.

D. ChromaDB

ChromaDB is a local vector database running from the `chroma_db` folder. * **Collection Name:** `lexai_documents`. * **Metadata:** Every chunk is saved with metadata like `{"user_id": 1, "document_id": "uuid"}`. This ensures User A's searches never return User B's documents.

E. RAG in LexAI (Step-by-Step)

1. User types "What are my termination risks?"
2. `classify_intent` determines if it's Document-Specific or General Legal.
3. `embed_question()` converts the question into numbers.
4. ChromaDB searches `lexai_documents` and retrieves the 8 mathematically closest chunks, strictly filtering by `user_id`.
5. These 8 chunks are injected into a massive strict prompt (`prompt = f"""You are LexAI... RETRIEVED CONTEXT: {context_text}"""`).
6. Gemini reads the context and generates an answer strictly based on the provided text.

MODULE 3: COMPLETE PROJECT WORKFLOW

Workflow D: Document-Specific Chat (Trace)

1. **Frontend:** User clicks "Send" in `chatbot.html`.
2. **JS Fetch:** `chatbot.js` sends a POST request with JSON payload `{"message": "hi", "document_id": "123"}`.
3. **Flask Route:** `chat_routes.py` `@chat_bp.route('/api/chat')` receives it.
4. **Service Call:** Calls `answer_question(user_id, question, document_id)` in `rag_service.py`.
5. **ChromaDB:** Filters vectors where `document_id == 123` and `user_id == current_user.id`.
6. **Gemini:** Generates response.
7. **Database:** Route saves the exchange to the `Messages` table.
8. **Frontend:** JS receives JSON, renders Markdown using `marked.js`, and displays it.

MODULE 4: RUNNING ON WINDOWS

1. **Open PowerShell** in the project folder (`C:\Users\Rohan Bongane\...`).
2. **Activate Virtual Environment:** `.\venv\Scripts\activate`
3. **Install Dependencies:** `pip install -r requirements.txt`
4. **Check `.env`:** Ensure `SQLALCHEMY_DATABASE_URI=mysql+pymysql://root:password@localhost/lexai_db` is correct and `GEMINI_API_KEY` is set.

5. **Start MySQL:** Ensure your MySQL service (e.g., MySQL80) is running in Windows Services.
6. **Run Server:** `python app.py`. Wait for "Running on http://127.0.0.1:5000".
7. **Stop Server:** Press `CTRL + C`.

MODULE 5: TESTING AND PYTEST

LexAI has an excellent testing suite in the `tests/` folder (`test_rag.py`, `test_security.py`). * **Run Tests:** `pytest tests/` * **Fixtures:** `conftest.py` sets up a temporary database in memory so testing doesn't destroy your real MySQL data. * **Mocking:** Tests use `unittest.mock` to simulate Gemini API responses so you don't burn through your API quota while testing.

MODULE 6: SECURITY REVIEW

- **Verified Protection:** Passwords are mathematically hashed using `Flask-Bcrypt`.
- **Verified Protection:** IDOR Prevention. ChromaDB vector retrieval strictly filters by `{"user_id": {"$eq": int(user_id)}}`. A user cannot query another user's document vectors.
- **Verified Protection:** XSS Prevention. Frontend uses `DOMPurify.sanitize()` before injecting AI markdown into the chat UI.
- **Verified Protection:** Path Traversal. Uploads use `werkzeug.secure_filename` and absolute path checking.
- **Recommended Improvement:** Add rate-limiting (e.g., Flask-Limiter) to prevent users from spamming the Gemini API and exhausting quotas.

MODULE 7: TROUBLESHOOTING HANDBOOK

Symptom	Cause	Safe Fix
`Access denied for user 'root'@'localhost'`	Wrong MySQL password in `.env`.	Open `.env` and fix the `SQLALCHEMY_DATABASE_URI` password.
Chat replies: "AI service is busy"	Gemini quota hit or no internet.	Wait a minute. LexAI will automatically use the local KB fallback.
`ModuleNotFoundError`	Venv not active or missing package.	Run `venv\Scripts\activate` then `pip install -r requirements.txt`.
Port 5000 is in use	Another python/flask app is running.	Close old terminals or change `port=5001` in `app.py`.

MODULE 8: PRACTICAL LEARNING EXERCISES

Exercise 1: Locate where ChromaDB searches for documents. * *File:* `services/rag_service.py`. * *Task:* Find the `collection.query()` function and observe how `where_filter` locks the search to a specific `user_id`.

Exercise 2: Trace the PyMuPDF extraction. * *File:* `services/document_service.py`. * *Task:* Find `extract_pdf_text()`. See how it iterates over pages and strips white spaces using `clean_text()`.

MODULE 9: 60 MCA VIVA QUESTIONS & ANSWERS

(Questions 1-20: Database & Architecture) 1. **What is an ORM?** Object Relational Mapper. It lets us interact with MySQL using Python objects instead of raw SQL strings. LexAI uses Flask-SQLAlchemy. 2. **What is a Foreign Key in LexAI?** In the `Documents` table, `user_id` is a foreign key pointing to the `Users` table. It ensures every document belongs to a valid user. 3. **What is the difference between SQLite and MySQL?** SQLite is file-based and used for testing/small apps. MySQL is a robust, concurrent relational database server used in production. LexAI uses MySQL. 4. **How do you prevent SQL Injection?** By using an ORM like SQLAlchemy, which automatically sanitizes inputs and

uses parameterized queries. 5. **How are passwords secured?** Using `Flask-Bcrypt`, which hashes and salts the passwords so they are mathematically impossible to reverse-engineer. 6. **What is Jinja2?** The templating engine Flask uses to dynamically generate HTML before sending it to the browser. 7. **Why use Blueprints in Flask?** To modularize code. Instead of 100 routes in `app.py`, we split them into `auth_routes`, `chat_routes`, etc. 8. **What does `db.session.commit()` do?** It permanently saves the pending database transaction to MySQL. 9. **What happens if a database operation fails halfway?** We use `db.session.rollback()` inside a try/except block to undo changes and prevent database corruption. 10. **Explain a 1-to-Many relationship in your app.** One User can have Many Documents. 11. **Explain a 1-to-1 relationship in your app.** One Document has One DocumentAnalysis record. 12. **How does LexAI know who is logged in?** It uses `Flask-Login` which stores an encrypted session cookie in the user's browser. 13. **Where are the physical PDFs stored?** In the local file system under `uploads/<user_id>/<doc_id>/`. The database only stores the file path. 14. **Why not store PDFs inside MySQL?** Storing large binary files (BLOBs) slows down relational databases. File systems are optimized for files. 15. **What is a Primary Key?** A unique identifier for a row, like `id` in the `users` table. 16. **What does `secure_filename()` do?** It prevents directory traversal attacks by removing slashes and dangerous characters from uploaded filenames. 17. **What is `.env` used for?** Storing sensitive configuration like DB passwords and API keys outside of the source code. 18. **Why should `.env` be in `.gitignore`?** So you don't accidentally publish your secrets to GitHub. 19. **What is the entry point of your application?** `app.py`. 20. **How does the frontend communicate with the chat API?** Using JavaScript `fetch()` API to send asynchronous POST requests with JSON payloads.

(Questions 21-40: AI, Embeddings, ChromaDB & RAG) 21. **What is an Embedding?** A numerical array representing the semantic meaning of text. 22. **Which embedding model does LexAI use?** `SentenceTransformer('all-MiniLM-L6-v2')`. 23. **Why do you chunk documents?** Because LLMs have token limits, and embedding an entire 100-page book at once dilutes the mathematical semantic meaning. 24. **What is your chunk size?** 1000 words with a 150-word overlap. 25. **Why overlap chunks?** To ensure a sentence or concept split across two chunks doesn't lose its context. 26. **What is ChromaDB?** An open-source vector database optimized for storing embeddings and performing similarity searches. 27. **What is Semantic Search?** Searching by meaning rather than exact keyword matches (e.g., finding "car" when you search for "automobile"). 28. **What metric does ChromaDB use to find similarity?** Cosine distance/similarity or L2 (Euclidean) distance. 29. **What is RAG?** Retrieval-Augmented Generation. We *retrieve* facts from ChromaDB, *augment* the prompt, and *generate* an answer with Gemini. 30. **Why use RAG instead of just asking Gemini?** Because Gemini hasn't read the user's private contract, and it might hallucinate legal facts. RAG grounds the AI in reality. 31. **Does RAG eliminate hallucinations?** No, but it significantly reduces them by forcing the AI to stick to the provided context blocks. 32. **How does LexAI keep User A from seeing User B's documents in the AI chat?** During ChromaDB queries, we enforce a strict metadata filter:

`where={"user_id": {"$eq": int(user_id)}}`. 33. **Which library extracts PDF text?** PyMuPDF. 34. **Which library extracts Word document text?** `python-docx`. 35. **What is Intent Classification?** LexAI uses Gemini to quickly categorize a user's question as either "General Legal" or "Document Specific" before searching ChromaDB. 36. **What happens if Gemini API goes down?** The `try/except` block catches the error and executes `construct_fallback_response()`, returning raw Knowledge Base text. 37. **What is a System Prompt?** Hidden instructions sent to the AI outlining its persona (e.g., "You are LexAI, an AI legal assistant..."). 38. **How do you ensure Gemini doesn't give legally binding advice?** We explicitly command it in the system prompt and append a disclaimer to every response. 39. **Where does the Knowledge Base data come from?** It is curated and added manually by administrators in the `KnowledgeBase` table. 40. **How is the KB separated from user documents in ChromaDB?** They use the metadata tag `"source_type": "knowledge_base"`.

(Questions 41-60: Testing, Debugging, and Complete Workflow) 41. **What is Pytest?** A Python testing framework used to write and run automated test cases. 42. **Why do we mock the Gemini API in tests?** To ensure tests run quickly, work without an internet connection, and don't consume paid API quotas. 43. **What does a 404 error mean?** The route or page was not found. 44. **What does a 500 error mean?** Internal Server Error. The Python code crashed. 45. **How do you debug a 500 error?** Look at the Python traceback in the terminal to see exactly which line of code failed. 46. **What is a Virtual Environment?** An isolated container for Python dependencies so LexAI's libraries don't clash with other projects on the PC. 47. **What is AJAX?** Asynchronous JavaScript and XML. It allows updating parts of a web page without reloading the entire page (used in our chat). 48. **How do you prevent Cross-Site Scripting (XSS)?** By sanitizing user input and using `DOMPurify` on the frontend before rendering Markdown. 49. **What is CSRF?** Cross-Site Request Forgery. (If asked, note that Flask-WTF handles this for forms). 50. **What is IDOR?** Insecure Direct Object Reference. If a user changes a URL parameter from `doc_id=1` to `doc_id=2` and sees someone else's document. 51. **How did you prevent IDOR?** Every document route verifies `document.user_id == current_user.id`. 52. **Explain the entire upload-to-chat workflow in 30 seconds.** User uploads PDF. Flask extracts text, chunks it, and embeds it into ChromaDB. When the user asks a question, ChromaDB retrieves relevant chunks, sends them to Gemini inside a prompt, and Gemini generates a contextual answer. 53. **What is the biggest limitation of your project?** It currently lacks OCR (Optical Character Recognition), so it cannot read scanned image-based PDFs, only digital text PDFs. 54. **How would you improve this project?** I would integrate OCR, add regional language support (Hindi, Marathi), and migrate ChromaDB to a cloud vector store for horizontal scaling. 55. **How did you test your RAG pipeline?** By creating a test suite that uploads a mock contract, asks a specific question about it, and asserts that the response contains the expected legal clause. 56. **What is the role of `conftest.py`?** It sets up fixtures, like a mock database and test client, that can be shared across all test files. 57. **How do you handle rate limits from Google Gemini?** The backend checks for HTTP 429/503 errors and gracefully falls back to the deterministic

retrieval mode. 58. **What does the `@login_required` decorator do?** It intercepts the request and redirects the user to the login page if they don't have an active session cookie. 59. **Why is `__name__` passed to `Flask(__name__)`?** It tells Flask where the application is located on the filesystem so it knows where to find the `templates` and `static` folders. 60. **Can LexAI replace a lawyer?** No, it is an assistive tool for legal professionals and laypeople to quickly parse contracts. Final verification by a human is always mandatory.

MODULE 10: FINAL REVISION CHECKLIST

- [x] Database Schemas created (MySQL/Flask-SQLAlchemy).
- [x] Password Hashing active (Bcrypt).
- [x] Document Text Extraction active (PyMuPDF).
- [x] Embedding Generation active (all-MiniLM-L6-v2).
- [x] Vector Retrieval active & Isolated (ChromaDB).
- [x] AI Integration active (Google GenAI SDK).
- [x] Fallback Mechanism active (rag_service.py).
- [x] Testing Suite available (`pytest tests/`).

Generated from verified source code analysis on local directory.